

汽车销售管理系统（SSM + Vue3）

day02-数据库设计、逆向工程与车辆列表-开发文档

预期成果：

- 9张数据库表创建完成，含测试数据
- MyBatis Generator生成实体类和Mapper
- 前端布局、路由、登录页完成
- 后端车辆分页接口可用
- 前端车辆列表页展示数据

一、今日任务概览

步骤	任务	负责人建议
1	执行建表SQL，创建9张表并插入测试数据	全体（后端优先）
2	配置MyBatis Generator，生成实体类和Mapper	后端同学
3	前端搭建MainLayout布局、路由、登录页	前端同学
4	后端开发车辆分页接口	后端同学
5	前端开发车辆列表页，调用接口展示数据	前端同学
6	联调测试，Git提交合并	全体

二、数据库设计（9张表）

2.1 执行建表SQL

打开MySQL客户端（命令行或IDEA Database工具），连接到 `car_sales` 数据库，执行以下SQL：

```
-- 如果表已存在，先删除
-- DROP TABLE IF EXISTS inventory_log, order_detail, sales_order, test_drive,
car, customer, employee, user, brand;

-- 1. 品牌表
CREATE TABLE brand (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL COMMENT '品牌名称',
  country VARCHAR(30) COMMENT '所属国家'
);
```

-- 2. 车辆表

```
CREATE TABLE car (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  brand_id INT NOT NULL COMMENT '品牌ID',  
  model VARCHAR(50) NOT NULL COMMENT '车型名称',  
  config VARCHAR(100) COMMENT '配置版本',  
  price DECIMAL(10,2) NOT NULL COMMENT '指导价(万元)',  
  stock INT NOT NULL DEFAULT 0 COMMENT '库存数量',  
  image_url VARCHAR(200) COMMENT '图片URL',  
  FOREIGN KEY (brand_id) REFERENCES brand(id)  
);
```

-- 3. 客户表

```
CREATE TABLE customer (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(30) NOT NULL COMMENT '客户姓名',  
  phone VARCHAR(15) NOT NULL COMMENT '联系电话',  
  address VARCHAR(100) COMMENT '地址',  
  create_time DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

-- 4. 员工表

```
CREATE TABLE employee (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(30) NOT NULL,  
  position VARCHAR(30) DEFAULT '销售顾问',  
  hire_date DATE  
);
```

-- 5. 用户认证表

```
CREATE TABLE user (  
  id INT PRIMARY KEY AUTO_INCREMENT COMMENT '用户ID',  
  username VARCHAR(50) UNIQUE NOT NULL COMMENT '登录用户名',  
  password VARCHAR(100) NOT NULL COMMENT '登录密码',  
  role VARCHAR(20) NOT NULL COMMENT '角色: admin/customer',  
  ref_id INT NOT NULL COMMENT '关联ID (employee.id 或 customer.id)',  
  create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间'  
);
```

-- 6. 试驾预约表

```
CREATE TABLE test_drive (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT NOT NULL COMMENT '客户ID',  
  car_id INT NOT NULL COMMENT '车辆ID',  
  test_time DATETIME NOT NULL COMMENT '预约时间',  
  status TINYINT DEFAULT 0 COMMENT '状态: 0待确认 1已完成 2已取消',  
  remark VARCHAR(200),  
  create_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (customer_id) REFERENCES customer(id),  
  FOREIGN KEY (car_id) REFERENCES car(id)  
);
```

-- 7. 销售订单表

```
CREATE TABLE sales_order (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  order_no VARCHAR(32) UNIQUE NOT NULL COMMENT '订单编号',  
  customer_id INT NOT NULL,  
  car_id INT NOT NULL,
```

```

    employee_id INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL COMMENT '成交单价',
    quantity INT DEFAULT 1 COMMENT '数量',
    total_amount DECIMAL(10,2) NOT NULL COMMENT '总金额',
    status TINYINT DEFAULT 0 COMMENT '0待付款 1已付款 2已取消',
    order_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customer(id),
    FOREIGN KEY (car_id) REFERENCES car(id),
    FOREIGN KEY (employee_id) REFERENCES employee(id)
);

```

-- 8. 订单明细表

```

CREATE TABLE order_detail (
    id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    insurance_info VARCHAR(100) COMMENT '保险信息',
    gift VARCHAR(100) COMMENT '赠品',
    FOREIGN KEY (order_id) REFERENCES sales_order(id)
);

```

-- 9. 库存日志表

```

CREATE TABLE inventory_log (
    id INT PRIMARY KEY AUTO_INCREMENT,
    car_id INT NOT NULL,
    change_type TINYINT NOT NULL COMMENT '1入库 2出库（销售）',
    change_quantity INT NOT NULL COMMENT '变更数量',
    before_stock INT NOT NULL,
    after_stock INT NOT NULL,
    remark VARCHAR(100),
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (car_id) REFERENCES car(id)
);

```

2.2 插入测试数据

-- 插入品牌

```

INSERT INTO brand (name, country) VALUES
('特斯拉', '美国'),
('比亚迪', '中国'),
('小米', '中国');

```

-- 插入车辆

```

INSERT INTO car (brand_id, model, config, price, stock, image_url) VALUES
(1, 'Model 3', '后驱版', 23.19, 15, 'https://example.com/tesla/model3.jpg'),
(1, 'Model Y', '长续航版', 29.99, 10, 'https://example.com/tesla/modely.jpg'),
(2, '汉EV', '千山翠限量版', 32.98, 8, 'https://example.com/byd/han.jpg'),
(3, 'SU7', 'Max创始版', 29.99, 12, 'https://example.com/xiaomi/su7.jpg'),
(1, 'Model 3', '长续航全轮驱动版', 27.19, 12, 'https://example.com/tesla/model3-longrange.jpg'),
(1, 'Model Y', '后驱版', 24.99, 10, 'https://example.com/tesla/modely-rwd.jpg'),
(1, 'Model Y', '高性能版', 36.39, 5, 'https://example.com/tesla/modely-performance.jpg'),
(2, '汉EV', '冠军版 506km', 20.98, 9, 'https://example.com/byd/han-ev.jpg'),
(2, '唐EV', '冠军版 600km', 24.98, 6, 'https://example.com/byd/tang-ev.jpg'),
(2, '海豹EV', '550km 标准版', 18.98, 10, 'https://example.com/byd/seal.jpg'),

```

```

(2, '海豚', '自由版', 11.68, 15, 'https://example.com/byd/dolphin.jpg'),
(3, 'SU7', '标准版', 21.59, 20, 'https://example.com/xiaomi/su7-standard.jpg'),
(3, 'SU7', 'Pro版', 24.59, 15, 'https://example.com/xiaomi/su7-pro.jpg'),
(3, 'SU7', 'Max版', 29.99, 10, 'https://example.com/xiaomi/su7-max.jpg'),
(3, 'SU7', 'Ultra版', 52.99, 3, 'https://example.com/xiaomi/su7-ultra.jpg');

-- 插入员工
INSERT INTO employee (name, position, hire_date) VALUES
('张伟', '销售顾问', '2024-01-15'),
('李娜', '销售顾问', '2024-03-01');

-- 插入客户
INSERT INTO customer (name, phone, address) VALUES
('王小明', '13800001111', '北京市海淀区'),
('赵丽', '13900002222', '上海市浦东新区');

-- 插入用户认证
INSERT INTO user (username, password, role, ref_id) VALUES
('admin', '123456', 'admin', 1),
('lina', '123456', 'admin', 2),
('wangming', '123456', 'customer', 1),
('zhaoli', '123456', 'customer', 2);

```

2.3 验证表

执行 `SHOW TABLES;` 应看到9张表。执行 `SELECT * FROM car;` 应看到15条车辆记录。

三、MyBatis Generator 逆向工程

3.1 添加插件依赖到 pom.xml

在 `<build><plugins>` 中添加:

```

<plugin>
  <groupId>org.mybatis.generator</groupId>
  <artifactId>mybatis-generator-maven-plugin</artifactId>
  <version>1.4.2</version>
  <configuration>

    <configurationFile>src/main/resources/generatorConfig.xml</configurationFile>
    <overwrite>true</overwrite>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.33</version>
    </dependency>
  </dependencies>
</plugin>

```

3.2 创建 generatorConfig.xml

在 `src/main/resources` 目录下新建文件 `generatorConfig.xml`，内容如下（注意修改数据库密码）：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE generatorConfiguration
    PUBLIC "-//mybatis.org//DTD MyBatis Generator Configuration 1.0//EN"
    "http://mybatis.org/dtd/mybatis-generator-config_1_0.dtd">
<generatorConfiguration>
    <context id="mysql" targetRuntime="MyBatis3">
        <!-- 1. 注释生成器（可选） -->
        <commentGenerator>
            <property name="suppressAllComments" value="true"/>
        </commentGenerator>

        <!-- 2. 数据库连接（必须，且放在 commentGenerator 之后，javaTypeResolver 之前） -->
        <jdbcConnection driverClass="com.mysql.cj.jdbc.Driver"
            connectionURL="jdbc:mysql://localhost:3306/car_sales?
useSSL=false&serverTimezone=UTC"
            userId="root"
            password="123456">
            <property name="nullCatalogMeansCurrent" value="true"/>
        </jdbcConnection>

        <!-- 3. Java类型解析器（可选） -->
        <javaTypeResolver>
            <property name="forceBigDecimals" value="false"/>
        </javaTypeResolver>

        <!-- 4. 实体类生成位置（必须） -->
        <javaModelGenerator targetPackage="com.sales.entity"
targetProject="src/main/java">
            <property name="enableSubPackages" value="true"/>
            <property name="trimStrings" value="true"/>
        </javaModelGenerator>

        <!-- 5. SQL映射文件位置（可选） -->
        <sqlMapGenerator targetPackage="mybatis"
targetProject="src/main/resources">
            <property name="enableSubPackages" value="true"/>
        </sqlMapGenerator>

        <!-- 6. Mapper接口生成位置（可选） -->
        <javaClientGenerator type="XMLMAPPER" targetPackage="com.sales.mapper"
targetProject="src/main/java">
            <property name="enableSubPackages" value="true"/>
        </javaClientGenerator>

        <!-- 7. 表配置 -->
        <table tableName="brand" domainObjectName="Brand"/>
        <table tableName="car" domainObjectName="Car"/>
        <table tableName="customer" domainObjectName="Customer"/>
        <table tableName="employee" domainObjectName="Employee"/>
        <table tableName="test_drive" domainObjectName="TestDrive"/>
        <table tableName="sales_order" domainObjectName="SalesOrder"/>
    </context>
</generatorConfiguration>
```

```

<table tableName="order_detail" domainObjectName="OrderDetail"/>
<table tableName="inventory_log" domainObjectName="InventoryLog"/>
<table tableName="user" domainObjectName="User"/>
</context>
</generatorConfiguration>

```

3.3 运行生成器

- **方式一 (IDEA)** : 右侧Maven面板 → Plugins → mybatis-generator → mybatis-generator:generate 双击。
- **方式二 (命令行)** : 在项目根目录执行 `mvn mybatis-generator:generate`

生成成功后, 你会在项目中看到:

- `com.sales.entity` 包下: 9个实体类 (Brand.java, Car.java, ...)
- `com.sales.mapper` 包下: 9个Mapper接口 (BrandMapper.java, ...)
- `src/main/resources/mybatis` 下: 9个XML映射文件 (BrandMapper.xml, ...)

3.4 验证生成结果

打开 `CarMapper.java`, 检查是否包含 `selectByPrimaryKey`、`insert`、`updateByPrimaryKey` 等方法。

打开 `Car.java`, 检查字段是否与数据库一致。

四、前端：布局、路由与登录页

4.1 删除默认组件

删除 `src/components/HelloWorld.vue` (不需要)。

4.2 创建 MainLayout.vue

在 `src/layout/MainLayout.vue` 中写入:

```

<template>
  <el-container style="height: 100vh">
    <!-- 侧边栏 -->
    <el-aside width="200px" style="background: #001529">
      <div style="color:white; text-align:center; padding:20px 0; font-size:18px; font-weight:bold;">
        汽车销售
      </div>
    <el-menu
      :default-active="$route.path"
      router
      background-color="#001529"
      text-color="#fff"
      active-text-color="#ffd04b"
    >

```

```

    <el-menu-item index="/cars">车辆管理</el-menu-item>
    <el-menu-item index="/testdrive">试驾预约</el-menu-item>
    <el-menu-item index="/orders">订单管理</el-menu-item>
    <el-menu-item index="/stats">数据统计</el-menu-item>
  </el-menu>
</el-aside>

<!-- 主区域 -->
<el-container>
  <el-header style="background:white; display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid #ddd;">
    <span>欢迎使用汽车销售系统</span>
    <span>
      <el-tag type="info">管理员</el-tag>
      <el-button type="text" style="margin-left:10px;" @click="logout">退出
    </el-button>
  </el-header>
  <el-main>
    <router-view />
  </el-main>
</el-container>
</el-container>
</template>

<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()
const logout = () => {
  localStorage.removeItem('token')
  localStorage.removeItem('role')
  router.push('/login')
}
</script>

```

4.3 创建占位页面

在 `src/views` 下创建以下目录和文件（每个文件暂时只写一行文字）：

- `car/CarList.vue`
- `testdrive/TestDriveForm.vue`
- `order/OrderList.vue`
- `dashboard/Statistics.vue`

例如 `CarList.vue` 内容：

```

<template>
  <div>车辆管理页面（待开发）</div>
</template>

```

4.4 配置路由 (src/router/index.js)

```

import { createRouter, createWebHistory } from 'vue-router'

const routes = [
  {
    path: '/login',
    component: () => import('../views/Login.vue')
  },
  {
    path: '/',
    component: () => import('../layout/MainLayout.vue'),
    redirect: '/cars',
    children: [
      { path: '/cars', component: () => import('../views/car/CarList.vue') },
      { path: '/testdrive', component: () =>
import('../views/testdrive/TestDriveForm.vue') },
      { path: '/orders', component: () => import('../views/order/OrderList.vue') }
    ],
    { path: '/stats', component: () =>
import('../views/dashboard/Statistics.vue') }
  ]
}
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

// 路由守卫: 未登录跳转登录页
router.beforeEach((to, from, next) => {
  const token = localStorage.getItem('token')
  if (to.path !== '/login' && !token) {
    next('/login')
  } else {
    next()
  }
})

export default router

```

4.5 创建登录页 (src/views/Login.vue)

```

<template>
  <div style="height:100vh; display:flex; justify-content:center; align-
items:center; background:#f0f2f5;">
    <el-card style="width:380px; padding:20px;">
      <h2 style="text-align:center;">汽车销售系统</h2>
      <el-form :model="form" label-width="0">
        <el-form-item>
          <el-input v-model="form.username" placeholder="请输入用户名" prefix-
icon="User" />
        </el-form-item>
        <el-form-item>
          <el-input v-model="form.password" type="password" placeholder="请输入密
码" prefix-icon="Lock" />

```



```

        </el-form-item>
        <el-form-item>
          <el-button type="primary" style="width:100%;" @click="handleLogin">登录
        </el-button>
        </el-form-item>
        <div style="color:#999; font-size:12px; text-align:center;">默认账号:
admin / 密码: 123456</div>
      </el-form>
    </el-card>
  </div>
</template>

<script setup>
import { reactive } from 'vue'
import { useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'

const router = useRouter()
const form = reactive({ username: '', password: '' })

const handleLogin = () => {
  if (form.username === 'admin' && form.password === '123456') {
    localStorage.setItem('token', 'mock-token')
    localStorage.setItem('role', 'admin')
    ElMessage.success('登录成功')
    router.push('/')
  } else {
    ElMessage.error('用户名或密码错误')
  }
}
}
</script>

```

4.6 修改 src/App.vue

```

<template>
  <router-view />
</template>

```

4.7 测试前端

运行 `npm run dev`，访问 `http://localhost:5173`，应自动跳转到登录页。输入 admin/123456，登录后应看到侧边栏和“车辆管理页面（待开发）”。

五、后端：车辆分页接口

5.1 添加 PageHelper 分页依赖

如果 `pom.xml` 中已有，则跳过；否则添加：（已有，跳过）

```
<dependency>
  <groupId>com.github.pagehelper</groupId>
  <artifactId>pagehelper</artifactId>
  <version>5.3.2</version>
</dependency>
```

5.2 配置 PageHelper

在 `com.sales.config` 包中，如果有 `MyBatisConfig` 类，添加如下配置（若脚手架已配置则忽略）
（已将 `PageInterceptor` 注入到 `SqlSessionFactory` 中。这是 **MyBatis 官方标准方式**，与 Spring 整合更干净，跳过）：

```
@Bean
public PageHelper pageHelper() {
    PageHelper pageHelper = new PageHelper();
    Properties props = new Properties();
    props.setProperty("helperDialect", "mysql");
    props.setProperty("reasonable", "true");
    pageHelper.setProperties(props);
    return pageHelper;
}
```

5.3 编写 CarController

创建 `com.sales.controller.CarController`：

```
package com.sales.controller;

import com.github.pagehelper.PageInfo;
import com.sales.common.PageResult;
import com.sales.common.Result;
import com.sales.entity.Car;
import com.sales.service.CarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.math.BigDecimal;

@RestController
@RequestMapping("/car")
public class CarController {

    @Autowired
    private CarService carService;

    @GetMapping("/list")
    public Result list(@RequestParam(defaultValue = "1") int pageNum,
                      @RequestParam(defaultValue = "10") int pageSize,
```

```

        @RequestParam(required = false) Integer brandId,
        @RequestParam(required = false) BigDecimal minPrice,
        @RequestParam(required = false) BigDecimal maxPrice) {

    // 调用 Service 层
    PageInfo<Car> pageInfo = carService.findByPage(pageNum, pageSize,
brandId, minPrice, maxPrice);

    return Result.success(new PageResult(pageInfo.getTotal(),
pageInfo.getList()));
}
}

```

5.4 编写 CarService

创建 `com.sales.service.CarService`：

```

package com.sales.service;

import com.sales.entity.Car;
import com.github.pagehelper.PageInfo;

import java.math.BigDecimal;

public interface CarService {
    /**
     * 分页查询车辆（含品牌、价格筛选）
     */
    PageInfo<Car> findByPage(int pageNum, int pageSize, Integer brandId,
BigDecimal minPrice, BigDecimal maxPrice);
}

```

5.5 编写 CarServiceImpl

创建 `com.sales.service.service.CarServiceImpl`：

```

package com.sales.service.impl;

import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;
import com.sales.entity.Car;
import com.sales.entity.CarExample;
import com.sales.mapper.CarMapper;
import com.sales.service.CarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.util.List;

@Service
public class CarServiceImpl implements CarService {

```

```

@Autowired
private CarMapper carMapper;

@Override
public PageInfo<Car> findByPage(int pageNum, int pageSize, Integer brandId,
BigDecimal minPrice, BigDecimal maxPrice) {
    // 1. 启动分页
    PageHelper.startPage(pageNum, pageSize);

    // 2. 构建查询条件（使用 MBG 生成的 Example）
    CarExample example = new CarExample();
    CarExample.Criteria criteria = example.createCriteria();

    if (brandId != null) {
        criteria.andBrandIdEqualTo(brandId);
    }
    if (minPrice != null) {
        criteria.andPriceGreaterThanOrEqualTo(minPrice);
    }
    if (maxPrice != null) {
        criteria.andPriceLessThanOrEqualTo(maxPrice);
    }

    // 3. 执行查询（按创建时间倒序，保持列表一致性）
    example.setOrderByClause("id DESC");
    List<Car> list = carMapper.selectByExample(example);

    // 4. 返回分页对象
    return new PageInfo<>(list);
}
}

```

5.6 启动后端

在 IDEA 中配置 Tomcat（或使用内置服务器），启动后端，确保监听端口 8080。

5.7 测试接口

使用 Postman 或浏览器访问：

```
GET http://localhost:8080/car/list?pageNum=1&pageSize=10
```

应返回 JSON 格式数据，例如：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "total": 4,
    "rows": [...]
  }
}
```

六、前端：车辆列表页

6.1 封装车辆API

创建 `src/api/car.js`：

```
import request from '../utils/request'

export function getCarList(params) {
  return request({
    url: '/car/list',
    method: 'get',
    params
  })
}
```

6.2 完善 CarList.vue

替换 `src/views/car/CarList.vue` 内容为：

```
<template>
  <div>
    <h3>车辆管理</h3>
    <!-- 搜索表单 -->
    <el-form :inline="true" :model="searchForm" style="margin:10px 0;">
      <el-form-item label="品牌" >
        <el-select v-model="searchForm.brandId" placeholder="请选择品牌" clearable
style="width: 150px;" >
          <el-option label="特斯拉" value="1" />
          <el-option label="比亚迪" value="2" />
          <el-option label="小米" value="3" />
        </el-select>
      </el-form-item>
      <el-form-item label="价格区间">
        <el-input v-model="searchForm.minPrice" placeholder="最低价"
style="width:100px;" />
        ~
        <el-input v-model="searchForm.maxPrice" placeholder="最高价"
style="width:100px;" />
      </el-form-item>
      <el-form-item>
        <el-button type="primary" @click="search">搜索</el-button>
      </el-form-item>
    </el-form>
  </div>
</template>
```

```

        <el-button @click="reset">重置</el-button>
      </el-form-item>
    </el-form>

    <!-- 表格 -->
    <el-table :data="tableData" border>
      <el-table-column prop="id" label="ID" width="60" />
      <el-table-column prop="brandId" label="品牌ID" width="80" />
      <el-table-column prop="model" label="车型" />
      <el-table-column prop="config" label="配置" />
      <el-table-column prop="price" label="价格(万元)" width="120" />
      <el-table-column prop="stock" label="库存" width="80" />
    </el-table>

    <!-- 分页 -->
    <el-pagination
      v-model:current-page="pageNum"
      v-model:page-size="pageSize"
      :total="total"
      layout="total, prev, pager, next"
      @current-change="loadData"
      style="margin-top:15px;"
    />
  </div>
</template>

<script setup>
import { reactive, ref, onMounted } from 'vue'
import { getCarList } from '../api/car'

const searchForm = reactive({ brandId: '', minPrice: '', maxPrice: '' })
const tableData = ref([])
const total = ref(0)
const pageNum = ref(1)
const pageSize = ref(10)

const loadData = async () => {
  try {
    const params = {
      pageNum: pageNum.value,
      pageSize: pageSize.value,
      ...searchForm
    }
    const res = await getCarList(params)
    if (res.code === 200) {
      tableData.value = res.data.rows || []
      total.value = res.data.total || 0
    } else {
      console.error('加载失败', res.message)
    }
  } catch (error) {
    console.error('请求异常', error)
  }
}

const search = () => { pageNum.value = 1; loadData() }
const reset = () => {
  searchForm.brandId = ''

```

```
searchForm.minPrice = ''
searchForm.maxPrice = ''
search()
}

onMounted(() => { loadData() })
</script>
```

6.3 测试

确保后端已启动，前端 `npm run dev` 后登录，点击“车辆管理”，应显示车辆列表数据。

七、联调与Git提交

7.1 Git提交

每位成员在自己的功能分支上提交代码：

```
git add .
git commit -m "feat: 完成车辆分页接口"      # 后端同学
git commit -m "feat: 完成车辆列表页 + 布局"  # 前端同学
git push origin feature/xxxx
```

组长将各组员分支合并到 `dev`：

```
git checkout dev
git pull origin dev
git merge feature/backend-car
git merge feature/frontend-car
# 若有冲突，手动解决后提交
git push origin dev
```

所有同学拉取最新 `dev`：

```
git checkout dev
git pull origin dev
```

八、常见问题与解决方案

问题	解决方法
MBG生成失败，报找不到表	检查数据库名、表名大小写（Linux下区分大小写），检查 <code>generatorConfig.xml</code> 中的连接URL和密码。
后端启动报 <code>NoSuchBeanDefinitionException</code>	检查是否添加了 <code>@Controller</code> 或 <code>@RestController</code> ，并确保扫描包路径正确。
前端代理不生效，请求404	重启前端服务；检查 <code>vite.config.js</code> 代理配置的 <code>target</code> 是否为 <code>http://localhost:8080</code> ；确认后端已启动。
车辆列表数据显示空白	F12查看网络请求，检查返回数据格式是否与前端期望一致（需包含 <code>data.rows</code> 和 <code>data.total</code> ）。
Git合并冲突	仔细阅读冲突文件，保留双方需要的代码，删除 <code><<<<<<<</code> ， <code>=====</code> ， <code>>>>>>>></code> 标记后提交。

九、今日自检清单

- 数据库中有9张表，且 `car` 表有测试数据
- `CarMapper` 等Mapper接口和XML文件已生成
- 前端登录后可看到左侧菜单，点击“车辆管理”能切换页面
- 后端 `GET /car/list` 接口返回正确JSON
- 前端车辆列表页显示车辆数据，分页可切换
- 代码已提交到个人分支并合并到 `dev`

十、课后预习

- 第3天将开发车辆的增删改查功能，请提前熟悉 Element Plus 的 Dialog 和 Form 组件。
- 后端同学预习 `@RestController` 的 `@PostMapping`、`@PutMapping`、`@DeleteMapping` 用法。

祝实训顺利！如遇问题，先在小组内讨论，或请教老师。